Capacitación — App JavaFX

Empresa de Capacitación

> **Condiciones de la práctica (igual que la evaluación)**
> Trabajo individual, sin internet ni IA · 3 bloques de tiempo · el programa debe compilar y ejecutarse · se entrega el proyecto Maven completo en un ZIP con tu nombre completo. Una solución que no compila no evidencia ningún indicador.

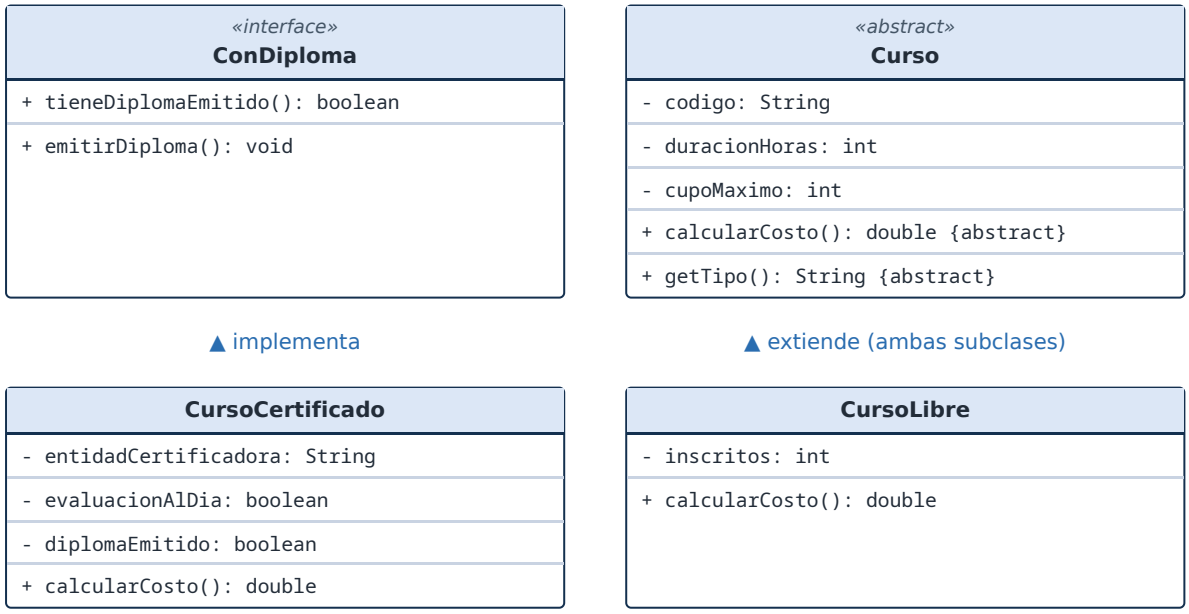
1. Contexto del caso

La empresa de capacitación ya cuenta con el modelo de clases desarrollado en la evaluación anterior: el sistema distingue cursos certificados y cursos libres, calcula el costo de cada curso según su tipo y permite marcar a los cursos certificados con diploma emitido. Sin embargo, ese sistema solo funciona por consola y la información se pierde cada vez que el programa se cierra.

La gerencia de la empresa ha solicitado transformarlo en una aplicación de escritorio de una sola ventana, que permita al personal de coordinación académica registrar y eliminar cursos, y que conserve la información en archivos para que esté disponible la próxima vez que se abra la aplicación.

Proyecto base

Partes desde **Capacitacion_Base.zip**: un proyecto Maven con la configuración inicial, la clase de arranque y el modelo ya implementado y probado, correspondiente al diagrama siguiente. Tu trabajo consiste en construir sobre ese modelo la interfaz gráfica y el almacenamiento en archivos. **No debes reescribir ni reemplazar las clases entregadas.**



El modelo entregado ya aplica las siguientes reglas:

- Curso Certificado: costo base de $85.000. Si la evaluación final no está al día, el costo se incrementa en un 20%.
- Curso Libre: costo base de $45.000. Si la cantidad de inscritos supera 20, el costo se incrementa en un 10%.

Datos comunes: el código no puede ser nulo ni vacío, la duración debe estar entre 4 y 200 horas y el cupo máximo debe ser mayor que cero. Si un dato no cumple, el modelo lo rechaza con un mensaje que explica el motivo.

2. Estructura del proyecto

La solución debe separar responsabilidades según el patrón **Modelo-Vista-Controlador**, con una capa de acceso a datos basada en el patrón **DAO/Repository**. El proyecto debe contener, como mínimo:

- **Configuración del proyecto:** el proyecto base ya incluye las dependencias de JavaFX y la clase de arranque de la aplicación. Debes agregar al archivo `pom.xml` la dependencia de la librería **Gson** disponible en el laboratorio, necesaria para trabajar con archivos JSON.
- **Distribución de paquetes y responsabilidades** acorde al patrón de diseño.

> **Reglas que la pauta exige**
> La interfaz se declara en **FXML** (no se acepta una interfaz construida íntegramente por código); la persistencia incluye una **interfaz DAO**, su implementación con Gson y una clase **Repository**, que trabajan con colecciones de dominio **sin depender de JavaFX**; la tabla se alimenta de una **colección observable**.

3. Ventana de la aplicación

Al ejecutar la aplicación se abre una única ventana, desde la cual se realizan todas las acciones. La ventana debe contener:

- Un **formulario de registro** con los campos descritos más abajo y un botón **Registrar**.
- Una **tabla** con los cursos registrados y las columnas Tipo, Código, Duración (h), Cupo y Costo curso. El costo que se muestra es el que calcula el modelo para cada curso.
- Un botón **Eliminar**, que quita del registro el curso seleccionado en la tabla. Si no hay una fila seleccionada, la aplicación informa que se debe seleccionar un curso y no realiza ningún cambio.
- Un **área de mensajes** dentro de la misma ventana, donde se informa el resultado de cada acción: registro exitoso, dato no válido, eliminación, falta de selección o error al leer o guardar los archivos. Cada mensaje reemplaza al anterior. **Los mensajes no se muestran por consola.**

Campos del formulario

- Un selector de tipo con las opciones Curso Certificado y Curso Libre.
- Datos comunes: código, duración en horas y cupo máximo.
- Datos del curso certificado: entidad certificadora, si la evaluación final está al día (sí/no) y si tiene diploma emitido (sí/no).
- Dato del curso libre: cantidad de inscritos.

Al cambiar el tipo en el selector, solo quedan habilitados los datos específicos del tipo elegido; los datos del otro tipo quedan deshabilitados y no se consideran al registrar.

Al presionar Registrar, la aplicación valida los datos según la sección 4. Si son correctos, registra el curso, la tabla lo muestra de inmediato, el área de mensajes informa la confirmación (por ejemplo: “CUR-C01 (Curso Certificado) registrado correctamente.”) y el formulario queda vacío para un nuevo registro.

4. Validaciones del formulario

Antes de registrar, el formulario debe verificar los datos ingresados según las siguientes reglas:

- **Tipo:** debe estar seleccionado.
- **Código:** no puede estar vacío ni contener solo espacios.
- **Duración:** número entero entre 4 y 200.
- **Cupo máximo:** número entero mayor que cero.
- **Entidad certificadora** (solo curso certificado): no puede estar vacía ni contener solo espacios.
- **Inscritos** (solo curso libre): número entero mayor o igual que cero.

Las reglas de código, duración y cupo máximo son las mismas que ya aplica el modelo entregado. Si algún dato no es válido:

- el curso no se registra;
- el área de mensajes indica **qué campo** es incorrecto y **por qué**;
- los valores ya ingresados se mantienen para que puedan corregirse;
- la aplicación no se detiene ni muestra errores por consola.

5. Almacenamiento de la información

La información de los cursos debe conservarse entre una ejecución y otra, en formato **JSON**, en la carpeta raíz del proyecto:

- `cursos_certificados.json`: contiene los cursos certificados registrados.
- `cursos_libres.json`: contiene los cursos libres registrados.

El comportamiento esperado es el siguiente:

- Al iniciar, la aplicación carga automáticamente la información de ambos archivos y la muestra en la tabla.
- Si los archivos no existen (por ejemplo, la primera vez que se ejecuta), la aplicación inicia con la tabla vacía y sin mostrar errores.
- Cada vez que se registra o se elimina un curso, los archivos se actualizan automáticamente, sin que el usuario deba presionar un botón para guardar.
- Lo que muestra la tabla y lo guardado en los archivos debe coincidir en todo momento.

6. Datos de prueba y resultado esperado

Registra desde la aplicación los siguientes cursos:

Tipo	Código	Duración (h)	Cupo	Datos específicos	Costo del curso
Curso Certificado	CUR-C01	40	25	Entidad: SENCE | Evaluación al día: no | Diploma: sí	$102.000
Curso Certificado	CUR-C02	60	20	Entidad: ChileValora | Evaluación al día: sí | Diploma: no	$85.000
Curso Libre	CUR-L01	20	30	Inscritos: 25	$49.500
Curso Libre	CUR-L02	16	15	Inscritos: 10	$45.000

Luego de registrarlos, la ventana debe verse de forma similar a la siguiente:

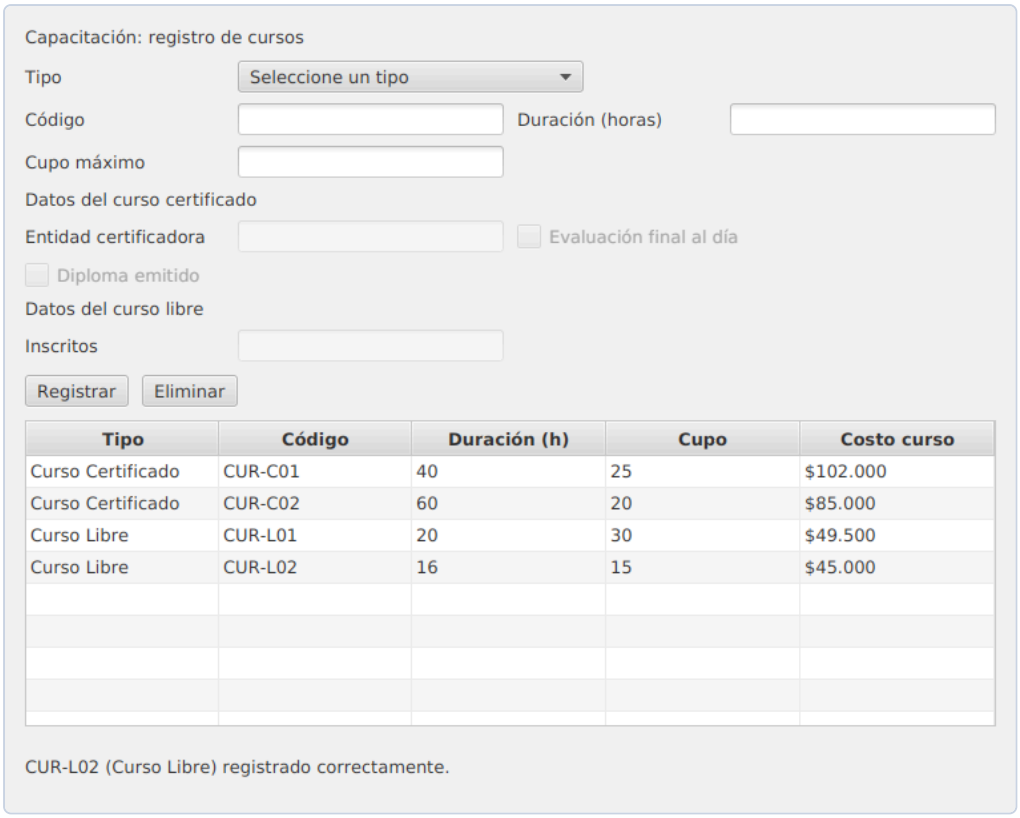


Figura 1. Ventana con los cuatro cursos registrados.

Si se presiona Eliminar sin seleccionar una fila, el aviso aparece en la misma ventana:

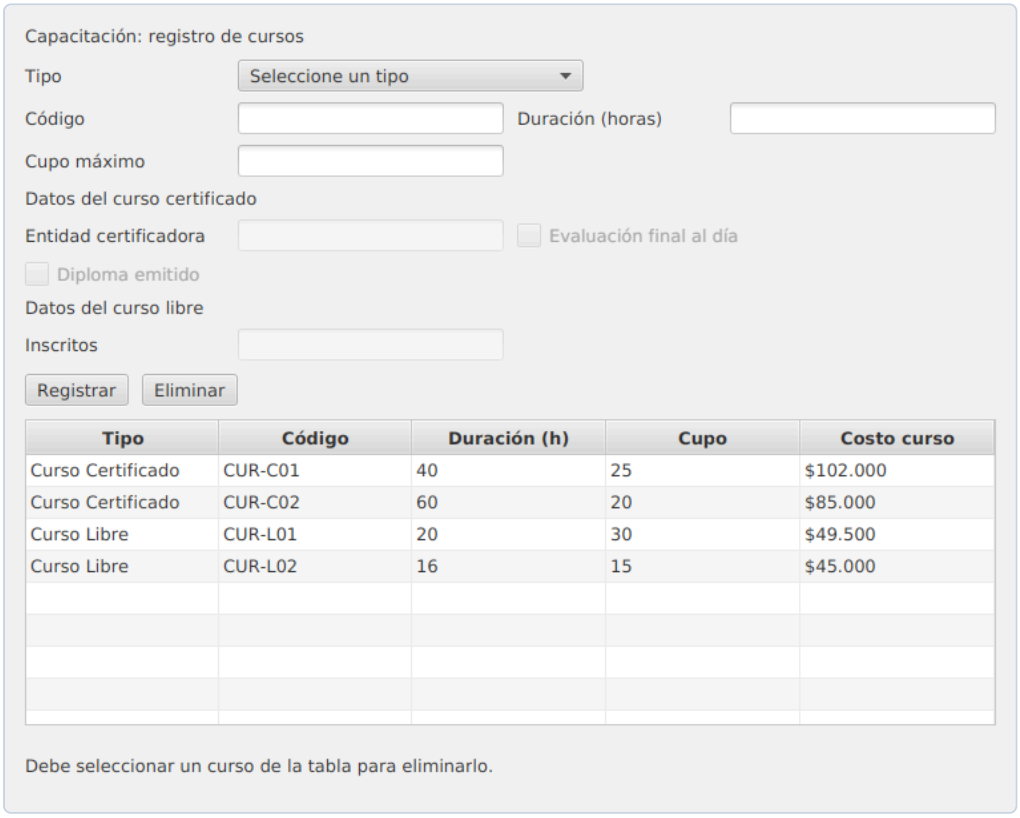


Figura 2. Aviso por falta de selección.

Si un dato del formulario no es válido, el área de mensajes indica el campo con problemas y se mantienen los valores ingresados:

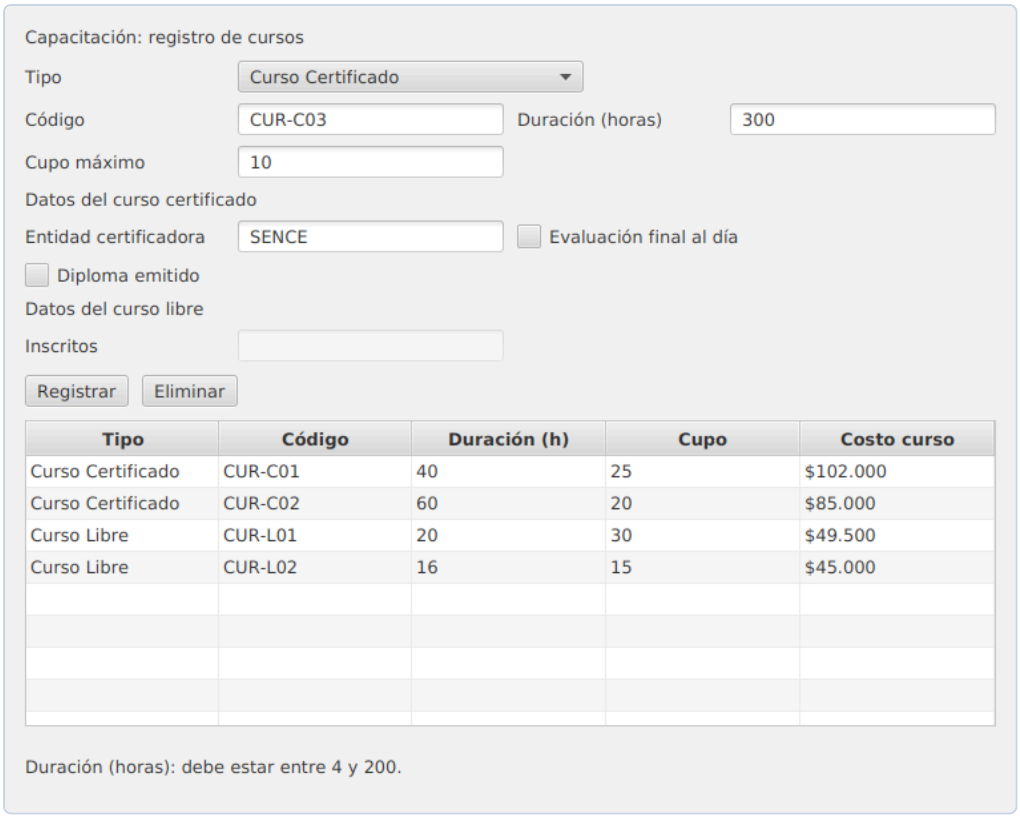


Figura 3. Error de validación en la duración.

Verificación de almacenamiento

- Cierra la aplicación y vuelve a abrirla: la tabla debe mostrar los mismos cuatro cursos.
- Elimina el curso CUR-C01, cierra la aplicación y vuelve a abrirla: CUR-C01 ya no debe aparecer.

Las imágenes son referenciales: la distribución, los textos y el estilo de las ventanas pueden variar, siempre que estén presentes todos los elementos solicitados y que los costos calculados sean correctos.